Week 08

중간 실기: Python 문제 해결

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

오늘의 진행

구간	시간	활동
1	15분	실기 안내, 문제 해결 5단계
2	90분	**중간 실기** (`assignment.ipynb`)
3	15분	제출 확인, 기술 메모 마무리

- 실기 전 주: 실습 노트북의 **빠른 점검 6문항 + 모의 실기**로 준비
- 실습 노트북 (복습 · 모의 실기)

1~7주차 지도

주차	핵심	실기에서 쓰는 곳
2	문자열 · f-string	보고서 문자열
3	조건 · 반복 · 경계값	등급 판정, 출석 규정
4	리스트 · 딕셔너리 · 정렬 key	학과별 요약, 상위 n명
5	함수 · 테스트 · raise	작은 함수로 나누기
6	CSV · 예외 처리	데이터 읽기와 검증
7	클래스	`GradeBook`

빠른 점검 6문항 (실습 노트북 1절)

문항	주차	내용
1	2주	`" 2026-09-07 "` → `(9, 7)`
2	3주	60점 이상 **3연속**의 위치
3	4주	`{학과: 최고점}`
4	5주	`"75%"` → `0.75`, 잘못된 형식은 `ValueError`
5	6주	CSV에서 `qty` 합계 (숫자 아닌 행 건너뛰기)
6	7주	`ParkingLot` 클래스 (만차 · 중복 입차)

> 정답을 지우고 **3분 안에 다시 쓸 수 있으면** 준비 완료

문제 해결 5단계

단계	할 일	시간 비율
① 요구사항 분석	입력 · 출력 · 규칙 · 예외를 **표로**	15%
② 설계	함수 / 클래스 목록과 책임	10%
③ 구현	작은 함수부터, 하나 만들 때마다 실행	45%
④ 테스트	정상 · 경계 · 오류, 자동 확인	20%
⑤ 기술 메모	설계 이유 · 테스트 · AI 사용 · 한계	10%

① 요구사항 표 (모의 실기 예)

구분	내용
입력	CSV (date, member, amount, memo), 회원 명단
출력	기록 리스트, 회원별 합계, 월별 미납자, 월별 수입
규칙	금액은 양의 정수, 날짜 앞 7글자가 월, 미납 = 그 달 합계 < 회비
예외	금액 오류 · 이름 빈칸 → 건너뛰고 개수 세기

> 규칙과 예외 **한 줄마다 테스트가 하나 이상** 있어야 한다

② 설계 → ③ 구현

```
def load_fees(path): ...                    # 6주차: 읽기 + 검증

class ClubLedger:                           # 7주차: 상태 + 집계
    def __init__(self, members, records): ...
    def paid_total(self, member): ...
    def unpaid(self, month, fee=10000): ...
    def monthly_income(self): ...

def report(ledger, months): ...             # 2주차: 출력 문자열
```

- 하나 만들 때마다 **가장 작은 입력으로 실행**
- 막히면 다음 문제로 — **부분 점수**가 있다

④ 테스트: 규칙마다 하나 이상

```
def test_ledger():
    lg = ClubLedger(MEMBERS, load_fees("club_fees.csv")[0])
    assert lg.paid_total("박지호") == 10000           # 나누어 낸 납부
    assert lg.unpaid("2026-03") == ["최하은"]
    assert lg.unpaid("2026-04") == ["박지호", "이서연"]  # 가나다순
    assert lg.unpaid("2026-05") == sorted(MEMBERS)     # 기록 없는 달
    assert lg.monthly_income() == {"2026-03": 30000, "2026-04": 40000}
```

⑤ 기술 메모

항목	좋은 예
설계	읽기 · 집계 · 출력을 분리해 각각 테스트할 수 있게 했다
테스트	나누어 낸 납부, 명단 밖 납부자, 기록 없는 달을 확인했다
AI 사용	월 정렬을 물어 `dict(sorted(...))` 채택, `except:` 는 `ValueError` 로 수정
한계	날짜 형식이 다르면 월 판단이 틀린다, 환불 처리 규칙이 없다

무엇을 했는지보다 **왜**, **무엇을 확인했고**, **무엇이 남았는지**

중간 실기 구성 (100점)

문제	내용	배점
1	`load_students` 데이터 읽기와 검증	20
2	`total_score` · `final_grade` · `dept_summary`	25
3	`GradeBook` 클래스 (`find` · `top` · `distribution` · `at_risk`)	25
4	`write_report` 보고서 파일	20
5	기술 메모	10

데이터: `students.csv` (80명 가상 데이터) · pandas 사용하지 않음

성적 규정 (공통)

항목	규정
총점	중간 30% + 기말 40% + 과제 30%, `round(x, 1)`
과제 미제출	빈 칸 → `None`, 총점 계산에서 0점
등급	90 A · 80 B · 70 C · 60 D · 그 외 F
출석	16회 중 **12회 미만이면 F**

> 경계: 총점 **90.0**, 출석 **12 / 11**, 과제 **None**

채점 방식

영역	비중	확인 방법
기능	60%	자동 확인 셀 + 채점자의 추가 입력 실행
코드 품질	20%	함수 분리, 이름, 중복, 예외 처리 방식
검증	10%	직접 추가한 경계 · 오류 확인
기술 메모	10%	설계 · 테스트 · AI 사용 · 한계

- 결과를 **하드코딩**한 코드 → 해당 문제 기능 점수 0
- `except:`로 모든 오류를 삼키는 코드 → 품질 감점

AI 사용 원칙 (실기)

- 사용 범위는 **교수자가 실기 당일 공지**한 대로 따른다
- 사용했다면 기술 메모에 **도구 · 질문 · 채택 · 수정**을 기록
- 기록과 코드가 **일치**해야 한다
- 제출한 코드는 **한 줄씩 설명할 수 있어야** 한다 (구술 확인 가능)

> AI가 만든 코드라도 책임은 **제출한 사람**에게 있다

시간 관리

시점	목표
0~10분	전체 문제 읽기, 요구사항 표 메모
10~35분	문제 1 (데이터 읽기) — 이후 문제의 기반
35~60분	문제 2
60~80분	문제 3
80~95분	문제 4, 기술 메모
95~105분	**다시 시작 및 모두 실행**, 제출

막혔을 때

1. 자동 확인 셀의 **기대값**을 다시 읽는다
2. **가장 작은 입력**(학생 1~2명)으로 직접 실행해 본다
3. 중간값을 `print` 해서 어디서 달라지는지 찾는다
4. 5분 이상 진전이 없으면 **다음 문제로**
5. 시간이 남으면 돌아온다

자주 하는 실수

실수	결과
CSV 값을 문자열 그대로 계산	`"80" * 0.3` → `TypeError`
과제 빈 칸을 `int("")`	`ValueError` → 정상 행을 건너뜀
출석 규정을 등급 계산 뒤에 확인	출석 미달인데 A
`sort()`로 입력 리스트 정렬	원본 변경
`distribution()`에 0명 등급 누락	키 개수 불일치
보고서에 숫자를 직접 입력	하드코딩 → 0점

제출 전 체크리스트

- [ ] **런타임 → 다시 시작 및 모두 실행** → 끝까지 오류 없음
- [ ] 자동 확인 결과(✅/❌)가 노트북에 남아 있음
- [ ] 결과를 직접 써 넣은 코드 없음
- [ ] 기술 메모 4항목 작성
- [ ] 파일 이름 `midterm_학번_이름.ipynb`

실기 후: 돌아보기

- 다음 주에 **모범답안 해설**과 흔한 오류를 함께 본다
- 자신의 코드와 비교해 **다르게 설계한 부분**을 찾아본다
- 9주차부터는 같은 일을 **NumPy · pandas**로 훨씬 짧게 하는 법을 배운다

```
# 9~10주차 예고: 오늘 수십 줄로 한 일을 몇 줄로
import pandas as pd
df = pd.read_csv("students.csv")
df.groupby("department")["final"].mean()
```

정리

오늘	핵심
문제 해결	요구사항 → 설계 → 구현 → 테스트 → 메모
통합	파일 · 예외 · 함수 · 자료구조 · 클래스를 하나로
검증	규칙마다 테스트, 경계값, 원본 불변
기술 메모	왜 · 무엇을 확인 · 무엇이 남았나

다음 주 · Week 09 NumPy와 배열 사고